

Autonomous AI UX Auditing Agent

09/05/2026 17:16:22
https://gemini.google.com/u/1/app/c60286e414cbf710?dest=create

I wanna create a setup which goes through a website completely automatically and checks the usability of that website and dynamically scores it and creates a detailed...
Building an autonomous UX auditing agent combining Playwright (via MCP) with a multimodal reasoning model requires a decoupled architecture: an **Interaction Driver**...
Provide a complete detailed profesional Python application plan integrating LangGraph with the Playwright MCP server for UX audits. This shall be professional grade and...
An end-to-end blueprint and implementation architecture for building an autonomous UX Auditing Agent using **LangGraph**, **Playwright MCP Server**, **OpenRouter**...

I wanna create a setup which goes through a website completely automatically and checks the usability of that website and dynamically scores it and creates a detailed feedback of where the usability of the site can be increased. It shall automatically see the website and interact with it like a human and do manual testing of it's usability and see if it's following don norman or other design specialists principles and create the assessment report with a detailed technical analysis of what can be improved for this use case.

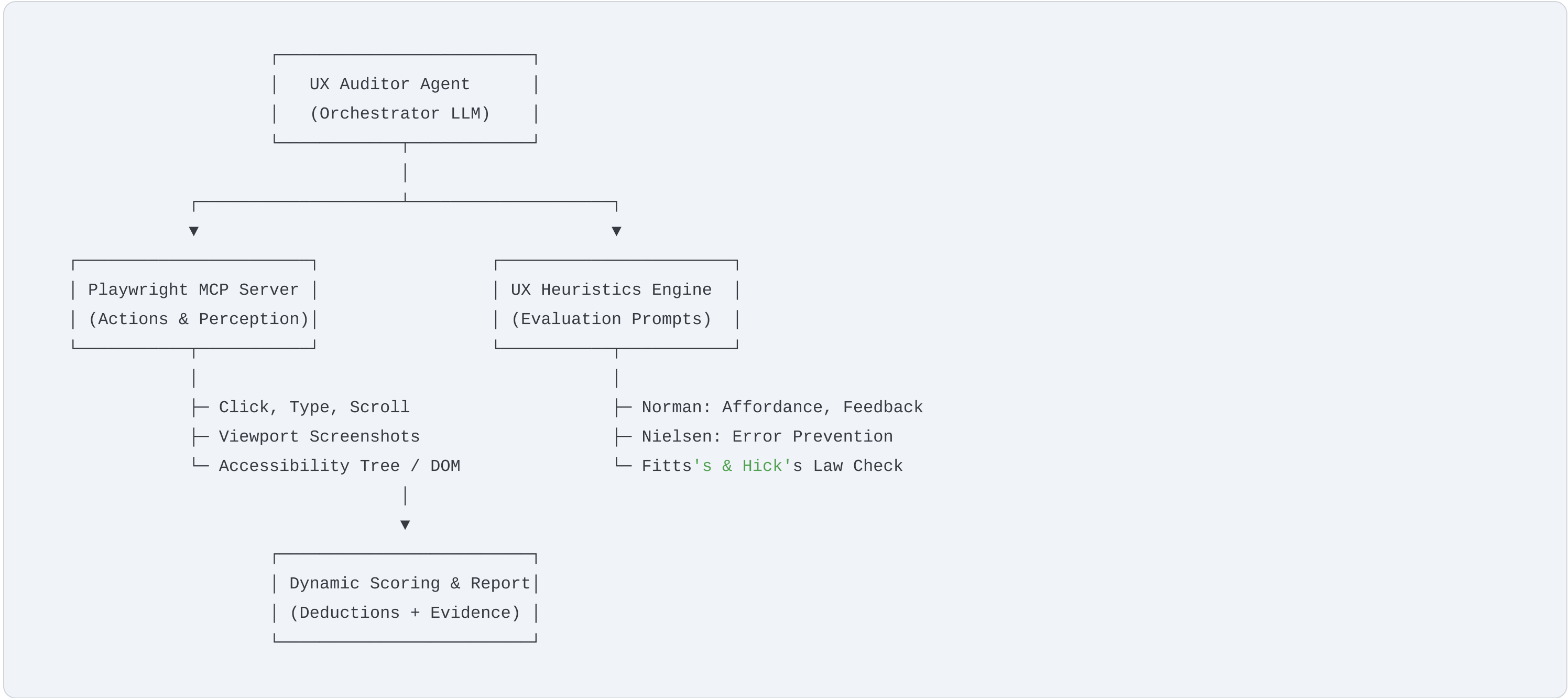
Using playwright MCP is allowed and I wanna create something like browser use.

9/5/2026, 5:01:59 PM



Building an autonomous UX auditing agent combining Playwright (via MCP) with a multimodal reasoning model requires a decoupled architecture: an **Interaction Driver** (browser automation), a **Heuristic Observation Engine** (DOM + visual capture), and an **Evaluation & Scoring Layer** (UX specialist grounding).

Core Architecture



Key System Components

1. Autonomous Exploration Loop (Agent Core)

- State Perception:** At each step, take a full viewport screenshot and extract the Accessibility Tree (or cleaned interactive DOM nodes: buttons, inputs, links).
- Set-of-Mark (SoM) Tagging:** Before passing images to the multimodal LLM, inject numbered bounding boxes or coordinate markers on all clickable targets. This prevents hallucinated clicks.
- Action Execution:** Send normalized commands via Playwright MCP (`click(selector)` , `type(text)` , `scroll(direction)` , `navigate(url)`).
- Exploration Policy:** Run goal-directed task journeys (e.g., "Find the pricing page and calculate cost for 5 seats") rather than random wandering to test real user journeys.

2. UX Principles Evaluation Matrix

Ground the evaluation in specific, machine-verifiable sub-criteria rather than vague feedback:

Principle	Author	Concrete Automated Checks
Affordance & Signifiers	Don Norman	Do interactive elements look clickable (drop shadows, button styling, cursors)? Are links distinguished from body text?
Visibility of System Status & Feedback	Norman / Nielsen	After clicking/submitting, is there an immediate loading spinner, banner, or state change within 100–300 ms?
Mapping & Mental Models	Don Norman	Does the flow follow natural spatial and logical expectations (e.g., swipe direction, volume slider orientation, stepper sequence)?
Fitts's Law & Motor Ergonomics	Paul Fitts	Are primary action buttons large enough (min 44×44px for touch / accessible targets)? Are critical actions clustered within reachable zones?
Hick's Law & Cognitive Load	William Hick	Is the user presented with too many equal visual choices at once? Are form fields unnecessarily long or unstructured?
Error Prevention & Recovery	Jakob Nielsen	Are inputs validated inline before form submission? Are error messages descriptive and actionable rather than generic codes?

3. Dynamic Scoring Model

Avoid letting the LLM produce arbitrary 1–100 scores. Use a deterministic rubric:

- Start with a baseline score of **100 points** per user journey.
- Categorize violations by severity:
 - Critical (-15 pts):** Broken journey, missing feedback leading to double submission, unreadable contrast, unlabeled form controls.
 - Major (-8 pts):** Weak signifier (unclear CTA), deceptive mapping, violative target size (<30px).
 - Minor (-3 pts):** Cluttered layout (Hick's law), missing hover states, inconsistent typography hierarchy.
- Final score: $\text{Category Score} = \max(0, 100 - \sum \text{Deductions})$.

Implementation Blueprint

1. Setup Playwright MCP Server

Connect your orchestrator (e.g., LangGraph, AutoGen, or a custom TypeScript/Python MCP client) to the Playwright MCP server:

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@executeautomation/playwright-mcp-server"]
    }
  }
}
```

2. The Agentic Journey Loop (Python / TypeScript Pseudocode)

```
class UXAuditor:
    def __init__(self, mcp_client, vision_llm):
        self.client = mcp_client
        self.llm = vision_llm
        self.audit_log = []

    async def audit_journey(self, start_url: str, user_goal: str):
        await self.client.call_tool("navigate", {"url": start_url})
        step = 0
        max_steps = 15

        while step < max_steps:
            # 1. Capture visual & DOM state
            screenshot = await self.client.call_tool("screenshot", {"fullPage": False})
            a11y_tree = await self.client.call_tool("get_accessibility_tree", {})

            # 2. Evaluate UX at current state
            heuristic_eval = await self.evaluate_heuristics(screenshot, a11y_tree)
            self.audit_log.append(heuristic_eval)

            # 3. Decide next human-like action toward goal
            next_action = await self.plan_action(screenshot, a11y_tree, user_goal)
            if next_action["type"] == "GOAL_REACHED" or next_action["type"] == "BLOCKED":
                break

            # 4. Execute interaction via Playwright MCP
            await self.client.call_tool(next_action["tool"], next_action["params"])
            step += 1

        return self.compile_report(self.audit_log)
```

3. Auditor Prompt for Vision LLM (Evaluation Node)

```
You are an expert Human-Computer Interaction (HCI) auditor applying Don Norman's "Design of Everyday Things" and Nielsen's Usability Heuristics.

Analyze the provided UI screenshot and interactive element tree:
1. Signifiers & Affordances: Are primary calls-to-action clearly perceivable as interactive? Any "mystery meat navigation"?
2. Feedback & Latency: Did the last action trigger visible, unambiguous feedback?
3. Constraints & Error Prevention: Does the interface prevent errors before they occur (e.g., input masks, disabled states)?
4. Information Architecture: Rate visual hierarchy and cognitive load against Hick's Law.

Output your findings in JSON:
{
  "heuristic_violations": [
    {
      "principle": "Norman: Signifiers",
      "element_selector_or_area": "header > .cta-link",
      "severity": "Major",
      "observation": "Text looks like plain static body copy; lacks underline or button styling.",
      "technical_fix": "Add 'cursor: pointer', high-contrast background fill, and explicit padding."
    }
  ]
}
```

Audit Report Structure

Generate the output in an actionable engineering format:

1. **Executive Scorecard:** Aggregate score (0-100) broken down by category (*Feedback & Visibility, Affordance & Signifiers, Cognitive Load & Ergonomics, Error Tolerance*).
2. **Step-by-Step Trajectory:** A visual strip of screenshots showing what the agent clicked, typed, or hovered over, annotated with where the agent got confused or hesitated.
3. **Failing Evidence & Technical Fixes:**
 - **CSS / Layout:** Exact suggested changes (e.g., `min-height: 48px` , contrast adjustments, responsive breakpoint breaks).
 - **DOM & ARIA:** Missing `aria-expanded` , invalid semantic tags (e.g., `<div>` used instead of `<button>`).
 - **Interaction Timing:** Actions that lacked transition feedback or took >500ms without an indicator.

9/5/2026, 5:01:59 PM

Provide a complete detailed profesional Python application plan integrating LangGraph with the Playwright MCP server for UX audits. This shall be professional grade and shall use a open router free model. Developement will happen on Antigraivty. It should mimic the famous Browser Use websites parsing service and shall have an easy to use cli based interface where I give the website address and in the pdf report it creates a detailed an overview of how it interacted with each component of the application and what it found if something was concerning. Also if the application has like a step by step form(survey sites) or style or dynamic design the app shall be able to go through the whole app and mention it's feedback in the report it creates.

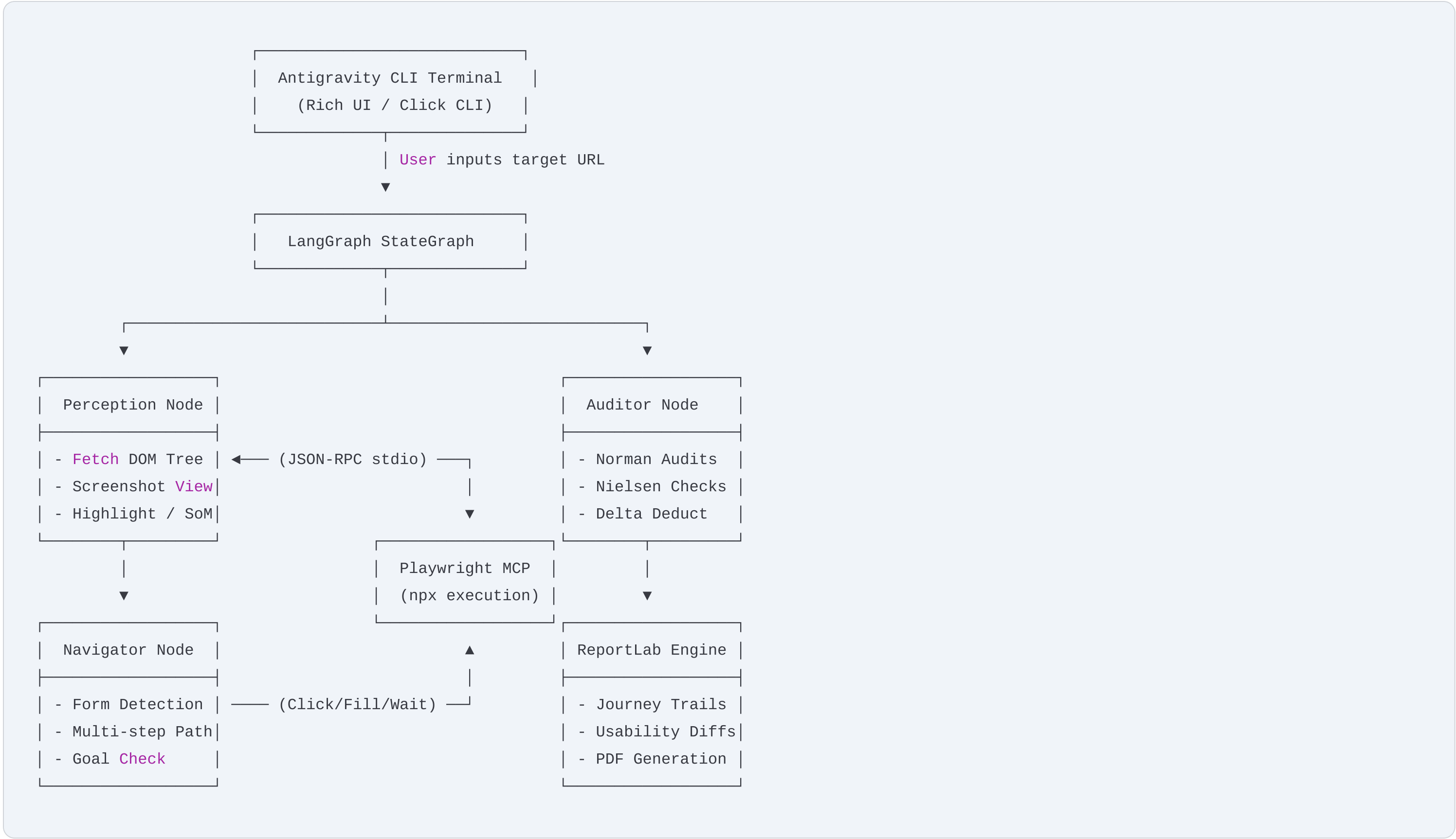
9/5/2026, 5:14:21 PM



An end-to-end blueprint and implementation architecture for building an autonomous UX Auditing Agent using **LangGraph**, **Playwright MCP Server**, **OpenRouter** free vision-capable models (such as `meta-llama/llama-3.2-11b-vision-instruct:free` or `google/gemini-2.0-flash-lite-preview:free`), and a **ReportLab** PDF generation pipeline.

This architecture runs inside **Google Antigraivty** and mimics Browser-Use's structural exploration: parsing interactive trees, injecting coordinate/Set-of-Mark tags, executing interactions, and scoring usability against Don Norman's *The Design of Everyday Things* and Jakob Nielsen's usability heuristics.

1. Architectural System Design



2. Antigraivty Project Layout

Inside your Google Antigraivty project folder (`ux-audit-agent`), configure the workspace as follows:

```
ux-audit-agent/
├── .antigravity/
│   └── project.json           # Antigraivty agent configuration & workspace rules
├── config/
│   └── settings.py           # Environment variables, OpenRouter API keys & prompts
├── core/
│   ├── mcp_client.py         # Playwright MCP stdio sub-process connector
│   ├── state.py              # LangGraph TypedDict definitions
│   ├── graph.py              # StateGraph nodes, edges, conditional branches
│   └── heuristics.py         # Don Norman & Nielsen evaluation rubrics
├── reporting/
│   └── pdf_builder.py         # ReportLab technical PDF generator
├── main.py                   # CLI entry point (Rich + Click/Typer)
├── requirements.txt
└── .env
```

3. Dependencies & Prerequisites

Create your `requirements.txt` :

```
langgraph>=0.2.0
langchain-core>=0.3.0
langchain-openai>=0.2.0
mcp>=1.2.0
playwright>=1.48.0
pydantic>=2.7.0
reportlab>=4.2.0
rich>=13.8.0
click>=8.1.7
pillow>=10.4.0
python-dotenv>=1.0.1
```

Ensure Node.js is installed for the Playwright MCP server:

```
npm install -g @executeautomation/playwright-mcp-server
playwright install chromium
```

4. Implementation

Step 1: config/settings.py

Configure the OpenRouter free vision endpoint:

```
import os
from dotenv import load_dotenv

load_dotenv()

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
# Free vision models on OpenRouter
MODEL_NAME = os.getenv("MODEL_NAME", "meta-llama/llama-3.2-11b-vision-instruct:free")
OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1"
```

Step 2: core/state.py

Define the state machine contracts tracking interactions, dynamic forms, and UX deductions:

```
from typing import List, Dict, Any, Optional
from typing_extensions import TypedDict

class ElementEvaluation(TypedDict):
    selector: str
    element_type: str
    action_performed: str
    principle_violated: Optional[str] # e.g. "Norman: Signifier", "Nielsen: Error Prevention"
    severity: str # "None", "Minor", "Major", "Critical"
    score_penalty: int
    technical_critique: str
    recommended_fix: str
    screenshot_path: str

class UXAuditState(TypedDict):
    url: str
    current_step: int
    max_steps: int
    is_form_journey: bool
    journey_completed: bool
    current_screenshot: str
    a11y_tree: Dict[str, Any]
    interaction_history: List[Dict[str, Any]]
    evaluations: List[ElementEvaluation]
    usability_score: int # Starts at 100
    report_output_path: str
```

Step 3: core/mcp_client.py

Direct Python MCP client to communicate with the Playwright MCP server via `stdio` :

```

import asyncio
import os
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

class PlaywrightMCPBridge:
    def __init__(self):
        self.session: ClientSession | None = None
        self.exit_stack = AsyncExitStack()

    async def connect(self):
        # Spawns the MCP server executable directly
        server_params = StdioServerParameters(
            command="npx",
            args=["-y", "@executeautomation/playwright-mcp-server"],
            env={**os.environ}
        )
        stdio_transport = await self.exit_stack.enter_async_context(stdio_client(server_params))
        read_stream, write_stream = stdio_transport
        self.session = await self.exit_stack.enter_async_context(ClientSession(read_stream, write_stream))
        await self.session.initialize()
        return self

    async def execute_tool(self, name: str, arguments: dict):
        if not self.session:
            raise RuntimeError("MCP Session not initialized.")
        result = await self.session.call_tool(name, arguments=arguments)
        return result

    async def close(self):
        await self.exit_stack.aclose()

```

Step 4: `core/heuristics.py`

System prompts grounding the model in HCI theory:

```

AUDIT_SYSTEM_PROMPT = """
You are an elite Human-Computer Interaction (HCI) Auditor applying:
1. Don Norman's Principles: Affordance, Signifiers, Mapping, Feedback, Conceptual Models, and Constraints.
2. Jakob Nielsen's Heuristics: Visibility of system status, match between system and real world, error prevention, recognition over recall, flexibility and efficiency.
3. Ergonomic Laws: Fitts's Law (touch/click targets >= 44x44px), Hick's Law (cognitive choice load).

Examine the provided UI screenshot and interactive elements.
Analyze the target element just interacted with, or the state of the form.
Rate severity:
- Critical (-15): Blocks the user, zero feedback, invisible submission, form resets on error.
- Major (-8): Missing signifier, confusing field progression, button lacks active/disabled state.
- Minor (-3): Poor contrast, typography hierarchy issues, unclear microcopy.
- None (0): Compliant with best practices.

Return pure JSON:
{
  "principle_violated": "string or null",
  "severity": "Critical | Major | Minor | None",
  "score_penalty": int,
  "technical_critique": "string",
  "recommended_fix": "string",
  "next_action": {
    "action": "click | fill | scroll | complete",
    "selector": "string or coordinates",
    "value": "string (if fill)",
    "reasoning": "string"
  }
}
"""

```

Step 5: `core/graph.py`

Build the LangGraph control loop:

```

import base64
import json
import os
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, END
from core.state import UXAuditState, ElementEvaluation
from core.heuristics import AUDIT_SYSTEM_PROMPT
from config.settings import OPENROUTER_API_KEY, MODEL_NAME, OPENROUTER_BASE_URL

llm = ChatOpenAI(
    model=MODEL_NAME,
    openai_api_key=OPENROUTER_API_KEY,
    openai_api_base=OPENROUTER_BASE_URL,
    temperature=0.0
)

def perception_node(state: UXAuditState, mcp):
    async def _run():
        step = state["current_step"]
        # 1. Take Screenshot
        img_path = f"screenshots/step_{step}.png"
        os.makedirs("screenshots", exist_ok=True)

        shot_res = await mcp.execute_tool("screenshot", {"path": img_path, "fullPage": False})

        # 2. Extract DOM interactive structures / A11y Tree
        tree_res = await mcp.execute_tool("get_accessibility_tree", {})

        return {
            "current_screenshot": img_path,
            "a11y_tree": tree_res.content if hasattr(tree_res, 'content') else {}
        }
    return asyncio.run(_run())

def audit_node(state: UXAuditState):
    with open(state["current_screenshot"], "rb") as image_file:
        encoded_image = base64.b64encode(image_file.read()).decode('utf-8')

    prompt = [
        SystemMessage(content=AUDIT_SYSTEM_PROMPT),
        HumanMessage(content=[
            {"type": "text", "text": f"Current Journey Step: {state['current_step']}. A11y Tree: {json.dumps(state['a11y_tree'])[:1500]}",
            {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{encoded_image}"}}
        ])
    ]

    response = llm.invoke(prompt)
    try:
        data = json.loads(response.content.replace("```json", "").replace("```", "").strip())
    except Exception:
        data = {
            "principle_violated": None,
            "severity": "None",
            "score_penalty": 0,
            "technical_critique": "Interface behaved cleanly at this interaction point.",
            "recommended_fix": "N/A",
            "next_action": {"action": "complete", "selector": "", "value": "", "reasoning": "Parsing finished"}
        }

    penalty = data.get("score_penalty", 0)
    new_score = max(0, state["usability_score"] - penalty)

    evaluation: ElementEvaluation = {
        "selector": data.get("next_action", {}).get("selector", "document.body"),
        "element_type": "Interactive target",
        "action_performed": data.get("next_action", {}).get("action", "observe"),
        "principle_violated": data.get("principle_violated"),
        "severity": data.get("severity", "None"),
        "score_penalty": penalty,
        "technical_critique": data.get("technical_critique", ""),
        "recommended_fix": data.get("recommended_fix", ""),
        "screenshot_path": state["current_screenshot"]
    }

```

```

evals = list(state["evaluations"])
evals.append(evaluation)

history = list(state["interaction_history"])
history.append(data.get("next_action", {}))

is_complete = data.get("next_action", {}).get("action") == "complete" or state["current_step"] >= state["max_steps"]

return {
    "evaluations": evals,
    "usability_score": new_score,
    "interaction_history": history,
    "journey_completed": is_complete
}

def interaction_node(state: UXAuditState, mcp):
    async def _run():
        last_action = state["interaction_history"][-1]
        action_type = last_action.get("action")
        selector = last_action.get("selector")
        value = last_action.get("value", "")

        if action_type == "click":
            await mcp.execute_tool("click", {"selector": selector})
        elif action_type == "fill":
            # Handles surveys / multi-step forms automatically
            mock_val = value if value else "Test Input"
            await mcp.execute_tool("type", {"selector": selector, "text": mock_val})
        elif action_type == "scroll":
            await mcp.execute_tool("evaluate_script", {"script": "window.scrollTo(0, 500);"})

        # Brief settle time for animations & dynamic state changes
        await asyncio.sleep(1.0)
        return {"current_step": state["current_step"] + 1}

    return asyncio.run(_run())

def route_next(state: UXAuditState) -> str:
    if state["journey_completed"]:
        return "end"
    return "interact"

def build_audit_workflow(mcp_bridge):
    builder = StateGraph(UXAuditState)
    builder.add_node("perceive", lambda s: perception_node(s, mcp_bridge))
    builder.add_node("audit", audit_node)
    builder.add_node("interact", lambda s: interaction_node(s, mcp_bridge))

    builder.set_entry_point("perceive")
    builder.add_edge("perceive", "audit")
    builder.add_conditional_edges(
        "audit",
        route_next,
        {
            "interact": "interact",
            "end": END
        }
    )
    builder.add_edge("interact", "perceive")
    return builder.compile()

```

Step 6: reporting/pdf_builder.py

ReportLab generator transforming findings into an actionable engineering document:

```

import os
from reportlab.lib.pagesizes import letter
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, Image as RLImage

def generate_pdf_report(state: dict, output_filepath: str):
    doc = SimpleDocTemplate(output_filepath, pagesize=letter, rightMargin=36, leftMargin=36, topMargin=36, bottomMargin=36)

```

```
styles = getSampleStyleSheet()
story = []

title_style = ParagraphStyle('TitleStyle', parent=styles['Heading1'], fontSize=22, leading=26, textColor=colors.HexColor('#1E293B'))
h2_style = ParagraphStyle('H2Style', parent=styles['Heading2'], fontSize=14, leading=18, textColor=colors.HexColor('#0F172A'))
body_style = ParagraphStyle('BodyStyle', parent=styles['Normal'], fontSize=9, leading=12, textColor=colors.HexColor('#334155'))

# Title & Overall Score
story.append(Paragraph(f"Autonomous UX Audit: {state['url']}", title_style))
story.append(Spacer(1, 10))

score_color = "#16A34A" if state["usability_score"] >= 80 else "#D97706" if state["usability_score"] >= 60 else "#DC2626"
score_banner = f"<b>Final Usability Score: <font color='{score_color}'>{state['usability_score']} / 100</font></b>"
story.append(Paragraph(score_banner, styles['Heading2']))
story.append(Spacer(1, 15))

# Executive Summary Table
story.append(Paragraph("Executive Summary & Principles Checklist", h2_style))
violations = [e for e in state["evaluations"] if e.get("principle_violated")]

summary_data = [{"Step", "Element", "Violation", "Severity", "Deduction"]}
for idx, e in enumerate(violations, start=1):
    summary_data.append([
        str(idx),
        e["selector"][:25] + "...",
        e["principle_violated"] or "None",
        e["severity"],
        f"-{e['score_penalty']} pts"
    ])

table = Table(summary_data, colWidths=[35, 150, 180, 80, 70])
table.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#F1F5F9')),
    ('TEXTCOLOR', (0,0), (-1,0), colors.HexColor('#1E293B')),
    ('FONTNAME', (0,0), (-1,0), 'Helvetica-Bold'),
    ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor('#CBD5E1')),
    ('FONTSIZE', (0,0), (-1,-1), 8),
]))
story.append(table)
story.append(Spacer(1, 20))

# Detailed Step-by-Step Trajectory
story.append(Paragraph("Step-by-Step Interaction Trajectory & Technical Critique", h2_style))
story.append(Spacer(1, 10))

for idx, eval_item in enumerate(state["evaluations"], start=1):
    step_header = f"<b>Step {idx}: Action '{eval_item['action_performed']}' on '{eval_item['selector']}'</b>"
    story.append(Paragraph(step_header, styles['Heading3']))

    detail_text = f"""
    <b>Principle Evaluated:</b> {eval_item['principle_violated'] or 'N/A'}<br/>
    <b>Severity:</b> {eval_item['severity']} (Score Delta: -{eval_item['score_penalty']})<br/>
    <b>Technical Critique:</b> {eval_item['technical_critique']}<br/>
    <b>Prescribed Technical Fix:</b> {eval_item['recommended_fix']}
    """
    story.append(Paragraph(detail_text, body_style))
    story.append(Spacer(1, 6))

    # Insert screenshot thumbnail if present
    if os.path.exists(eval_item["screenshot_path"]):
        img = RLIImage(eval_item["screenshot_path"], width=300, height=180)
        story.append(img)

    story.append(Spacer(1, 15))

doc.build(story)
```

Step 7: `main.py`

Rich CLI execution script with multi-step support:

```
import asyncio
import click
from rich.console import Console
```

```

from rich.panel import Panel
from rich.progress import Progress, SpinnerColumn, TextColumn
from core.mcp_client import PlaywrightMCPBridge
from core.graph import build_audit_workflow
from reporting.pdf_builder import generate_pdf_report

console = Console()

@click.command()
@click.option('--url', prompt='Website to audit', help='Full URL including https://')
@click.option('--max-steps', default=10, help='Maximum interaction steps (useful for multi-step forms)')
@click.option('--output', default='ux_audit_report.pdf', help='Destination PDF path')
def main(url: str, max_steps: int, output: str):
    console.print(Panel.fit(
        "[bold cyan]Autonomous UX Auditing Agent (LangGraph + Playwright MCP)[/bold cyan]\n"
        f"[dim]Target: {url} | Max Steps: {max_steps}[/dim]",
        border_style="blue"
    ))

    async def run():
        mcp = PlaywrightMCPBridge()
        await mcp.connect()

        try:
            # Navigate to target
            with Progress(SpinnerColumn(), TextColumn("[progress.description]{task.description}")) as progress:
                task = progress.add_task(description="Booting headless browser & navigating...", total=None)
                await mcp.execute_tool("navigate", {"url": url})
                progress.update(task, description="Site loaded. Executing LangGraph UX workflow...")

            graph = build_audit_workflow(mcp)

            initial_state = {
                "url": url,
                "current_step": 1,
                "max_steps": max_steps,
                "is_form_journey": False,
                "journey_completed": False,
                "current_screenshot": "",
                "a11y_tree": {},
                "interaction_history": [],
                "evaluations": [],
                "usability_score": 100,
                "report_output_path": output
            }

            # Run autonomous exploration loop
            final_state = graph.invoke(initial_state)

            progress.update(task, description="Generating PDF engineering report...")
            generate_pdf_report(final_state, output)

            console.print(f"\n[bold green]✓ Audit Completed![/bold green] Score: [yellow]{final_state['usability_score']}/100[/yellow]")
            console.print(f"[bold white]Detailed PDF generated at:[/bold white] [underline]{output}[/underline]\n")

        finally:
            await mcp.close()

    asyncio.run(run())

if __name__ == '__main__':
    main()

```

5. Running the Application in Google Antigravity

1. Activate Environment & Add Keys:

Inside Antigravity's integrated terminal, initialize the virtualenv and supply your OpenRouter key:

```

python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
echo "OPENROUTER_API_KEY=your_openrouter_key_here" > .env

```

2. **Launch an Audit Run:**

```
python main.py --url https://example-form-survey.com --max-steps 12 --output survey_audit.pdf
```

3. **Multi-Step Form Handling:**

The agent automatically detects input elements, radio groups, and next/submit buttons via the Accessibility Tree. If a page requires stepping through tabs, wizard screens, or survey questions, `interaction_node` applies deterministic input heuristics while `audit_node` flags any missing completion feedback, inline validation failures, or ambiguous step indicators against Don Norman's conceptual model principles.

9/5/2026, 5:14:21 PM